EXPEDIENTE: DETECTIVES DE AGENTES

Evaluación y observabilidad de sistemas de agentes

Tu agente dice que todo ha ido bien. ¿Cómo lo sabes?

Guía completa de la masterclass

David Rojas Cruz

Impartida el 5 de noviembre de 2026 · Bootcamp de Desarrollo de IA, Somos F5

Web interactiva: observabilidad-agentes.drojas-7u7.workers.dev



© 2026 David Rojas Cruz. Contenido bajo licencia CC BY-NC-ND 4.0: se puede compartir citando al autor, sin fines comerciales y sin modificaciones. Bita es un personaje original del autor. Las personas, empresas, aerolíneas y datos que aparecen son ficticios.

Índice

Parte I. El caso y su contexto

1. Por qué existe este tema 3
2. El caso del vuelo de 95 € 3
3. El mismo caso en cuatro épocas 4

Parte II. Conceptos y herramientas

4. Conceptos clave 5
5. Monitorizar frente a observar 5
6. Las tres pistas: logs, métricas y trazas 6
7. Cómo leer una traza 6

Parte III. Investigar y resolver

8. La investigación del caso de Catalina 8
9. Del hallazgo a la solución 9

Parte IV. Evaluar y mejorar

10. Evaluación 10
11. El ciclo completo 10
12. Riesgos y ética 11

Parte V. Cierre

13. Errores de concepto 12
14. Conclusiones 12
15. Para seguir aprendiendo 12

1. Por qué existe este tema

Hoy sabemos construir agentes de IA: programas que reciben un encargo, deciden por su cuenta qué pasos dar y usan herramientas para cumplirlo. Pero construir un agente es solo la mitad del trabajo. La otra mitad consiste en poder responder a tres preguntas:

- ¿Qué ha hecho exactamente el agente, paso a paso?
- ¿Lo ha hecho bien?
- Si lo ha hecho mal, ¿dónde y por qué?

Sin respuesta a esas preguntas, un agente es una caja negra, y ninguna empresa puede confiar en una caja negra que toma decisiones con dinero o datos de clientes. La **observabilidad** responde a las preguntas sobre qué pasó y dónde falló. La **evaluación** responde a si estuvo bien. La trazabilidad, el logging y el debugging son las herramientas concretas para conseguirlo.

> **Construir un agente es fácil. Saber qué está haciendo y si lo hace bien, no.**

2. El caso del vuelo de 95 €

Destino Directo es una agencia de viajes online (ficticia) que atiende a sus clientes con un agente de IA. Una clienta, Catalina, le escribe por el chat:



```
Catalina: Resérvame el vuelo más barato de Madrid a Barcelona
          para el viernes, que cueste menos de 100 euros.

Agente:   ¡Listo! Te he reservado el vuelo de AeroBrisa por 85 euros.
          Tu localizador es M3X8RT.
```

Poco después, a Catalina le llega el cargo en su tarjeta: **95 €**. El agente dijo que todo había ido bien. Ningún programa dio error. Y, sin embargo, algo falló.

¿Cómo averiguarlo? Preguntar al propio agente no sirve: si se equivocó una vez, puede equivocarse otra al explicarlo. Revisar el código tampoco: no hay ninguna línea rota, el fallo está en una decisión. Y volver a ejecutar el encargo no garantiza repetir el fallo, porque los agentes no siempre hacen lo mismo con la misma entrada. La única vía fiable es consultar un registro de todo lo que hizo el agente, siempre que alguien lo haya preparado para registrarlo.

> **Las palabras no son pruebas.**

3. El mismo caso en cuatro épocas

En 1960, el ingeniero Rudolf Kálmán definió un sistema como observable cuando se puede conocer su estado interno a partir de lo que sale de él. La idea pasó después al software. Veamos cómo se habría investigado la queja de Catalina a lo largo de los años.

Época	Cómo era la reserva	La prueba	La lección
Años 80 y 90	Una agencia física. La empleada apunta la reserva en una ficha de papel.	La ficha: dice 85 €; las tasas no se apuntaron.	Registrar permite reconstruir lo que pasó, pero solo sabes lo que alguien se acordó de apuntar.
Años 2000	Una web con un único programa que anota cada paso en un archivo.	Los logs: una línea con la reserva a 85 € y otra con el cobro de 95 €.	El registro se hace solo, y como el programa siempre hace lo mismo, el fallo se puede repetir.
Años 2010	Una aplicación dividida en servicios: búsqueda, reserva y cobro, cada uno con sus logs.	La traza: un identificador común une los registros de los tres servicios.	Hay que conectar los registros. Google lo describió en 2010 (Dapper) y en 2019 nació el estándar OpenTelemetry.
Desde 2023	Un agente de IA que decide por su cuenta: busca, elige, reserva y responde.	La traza del agente, con cada decisión y los datos que usó.	Hay que registrar decisiones, no solo acciones. Y volver a ejecutar no sirve.

> **El problema siempre ha sido el mismo: saber qué ha pasado. Lo que cambia es la prueba.**

4. Conceptos clave

- **Modelo de lenguaje (LLM).** Un programa que, dado un texto, escribe la continuación más probable. No consulta datos ciertos: predice palabras. Es el "cerebro" del agente.
- **Temperatura y no determinismo.** La temperatura introduce algo de azar al elegir cada palabra, de modo que el mismo encargo puede producir resultados distintos. Por eso no se puede reproducir un fallo a voluntad.
- **Tokens y coste.** El modelo trabaja con trozos de texto llamados tokens, y cada token cuesta dinero y tiempo. Si un agente se atasca repitiendo pasos, el coste se dispara.
- **Herramientas.** Funciones que el agente puede pedir que se ejecuten. El modelo pide; el programa ejecuta y le devuelve el resultado. El agente de Destino Directo usa dos: `buscar_vuelos` y `reservar_vuelo`.
- **Agente.** Un modelo de lenguaje en un bucle: piensa qué hacer, actúa con una herramienta, mira el resultado y vuelve a empezar hasta terminar. Cada vuelta del bucle es una decisión.
- **Estado.** Todo lo que el agente lleva acumulado en cada momento: el encargo, los resultados de las herramientas y sus decisiones.

Los agentes fallan de una forma distinta a los programas clásicos: no son deterministas, encadenan muchas decisiones (un error pequeño al principio puede estallar al final) y pueden fallar **sin dar ningún error**.

5. Monitorizar frente a observar

Monitorizar es vigilar lo que ya sabes que quieres vigilar: si un servicio está activo, cuánto tarda, cuántos errores hay. **Observar** es poder averiguar lo que no habías previsto: por qué una conversación costó el triple, o por qué el agente repitió la misma búsqueda siete veces.

El ejemplo del coche

El testigo del salpicadero se enciende y avisa de que algo va mal, pero no dice qué. El mecánico conecta un aparato de diagnóstico y lee lo que el coche fue registrando durante el viaje: la temperatura subió, el ventilador dejó de responder y el motor entró en modo de protección. Con esos datos encuentra la causa. Y solo puede hacerlo porque el coche estaba preparado para registrar esos datos.

- El testigo del salpicadero equivale a las **métricas y alertas**.
- Los datos que registra el coche equivalen a las **trazas y los logs**.
- El mecánico con su aparato eres tú haciendo **debugging**.
- Preparar el coche para registrar es **instrumentar** el agente.

Un sistema no es observable por sí solo: hay que **instrumentarlo**, es decir, prepararlo para que registre lo que hace. En la práctica suele ser poco código: en un agente hecho con LangGraph basta con configurar unas variables de entorno para registrar todo en LangSmith, y con OpenTelemetry se añade una librería.

6. Las tres pistas: logs, métricas y trazas

Para estudiarlas usamos a **Charo**, otra clienta que esa misma semana pidió un vuelo de Madrid a Sevilla, por menos de 90 €, al mismo agente que atendió a Catalina. A ella todo le salió bien: el agente le reservó AeroBrisa por 79 € (69 € más 10 € de tasas). Con un caso que funcionó aprendemos qué es lo normal.

Logs: ¿qué pasó en cada momento?

Anotaciones de algo que pasó en un momento concreto. En agentes se escriben estructuradas, con campos que se pueden filtrar:

```
+50 ms  INFO  chat modelo-demo-1           estado=ok
+1,3 s  INFO  execute_tool buscar_vuelos   estado=ok
+2,1 s  INFO  chat modelo-demo-1           estado=ok
+3 s    INFO  execute_tool reservar_vuelo  estado=ok
+3,6 s  INFO  chat modelo-demo-1           estado=ok
```

Métricas: ¿cuánto tardó, cuánto costó?

Números medidos a lo largo del tiempo. En la reserva de Charo: 4,1 segundos, 3165 tokens, 0,0036 € (con precios simulados), 5 pasos y 0 errores. Las métricas avisan de que algo va mal, pero no dicen por qué.

Trazas: ¿qué provocó qué?

El recorrido completo de una tarea, en orden y con su relación de causa y efecto. Es la pista más importante para investigar agentes.

> **Los logs son frases sueltas en un cuaderno; la traza es el capítulo completo.**

7. Cómo leer una traza

Una traza es un árbol de **spans**. Un span es un tramo de trabajo con nombre, duración, estado y los datos que recibió y devolvió. Hay tres tipos: el **agente** (la tarea entera, que es la raíz del árbol), el **modelo** (cada decisión) y la **herramienta** (cada acción). Cada paso cuelga del que lo provocó, y cada uno tiene un estado técnico: correcto, aviso o error.

```
Agente       Atiende el encargo de Charo                 4,1 s
├─ Modelo       Decide buscar vuelos                      1,2 s
├─ Herramienta  Busca los vuelos disponibles               780 ms
├─ Modelo       Elige el más barato y decide reservarlo    980 ms
├─ Herramienta  Reserva el vuelo                           560 ms
└─ Modelo       Escribe la respuesta final                 420 ms
```

Dentro de cada span están los datos. Por ejemplo, lo que devolvió la reserva de Charo:

```
{ "estado": "confirmada", "localizador": "K7Q2PZ", "codigo_vuelo": "AB0412",
  "precio_final": 79, "tasas": 10 }
```

Los nombres técnicos de los atributos, como `gen_ai.request.model` o `gen_ai.usage.input_tokens`, siguen las convenciones de OpenTelemetry para IA generativa: un vocabulario común que entienden herramientas como LangSmith, Langfuse o Arize Phoenix, que muestran este mismo árbol para cada ejecución.

8. La investigación del caso de Catalina

En una empresa real, la investigación empieza con una incidencia: atención al cliente recibe la queja y la abre con prioridad alta. El equipo de IA localiza la traza de la conversación de Catalina por su localizador, M3X8RT, y la investiga en cinco pasos.

Paso 1. Mirar las métricas

Métrica	Charo (salió bien)	Catalina (el caso)
Duración	4,1 s	4,1 s
Tokens	3165	3187
Coste	0,0036 €	0,0036 €
Pasos	5	5
Errores	0	0

Son prácticamente iguales. Las métricas avisan de problemas grandes, pero este fallo no se ve en los números.

Paso 2. Mirar los colores

Todos los spans de la traza de Catalina están en verde. Y aun así pagó de más. **Verde solo significa que el código no se rompió, no que el agente hiciera bien su trabajo.** A esto se le llama un falso éxito.

Paso 3. Buscar la pista

La búsqueda devolvió tres vuelos, con un campo llamado `precio`: NubeAir 120, VuelaSol 98 y AeroBrisa 85. El precio de 95 € aparece por primera vez en lo que devolvió la reserva:

```
{ "estado": "confirmada", "localizador": "M3X8RT", "codigo_vuelo": "AB0738",
  "precio_final": 95, "tasas": 10 }
```

Paso 4. Encontrar la decisión equivocada

El agente eligió bien el vuelo: AeroBrisa seguía siendo el más barato incluso con tasas. El problema es que la búsqueda daba precios sin tasas, y el modelo, en el último paso, tenía delante el precio final de 95 € y respondió 85. El fallo no está en una línea de código: está en una decisión.

Paso 5. ¿De quién es la culpa?

De los dos. La herramienta de búsqueda era ambigua (su campo `precio` no decía que era sin tasas) y el modelo no comprobó el dato que tenía delante.

> **No siempre la culpa es solo del modelo.**

9. Del hallazgo a la solución

Encontrar el fallo no es el final: es el principio de la solución. Investigar una traza sirve para reparar a la clienta, medir el impacto (cuántos clientes más sufrieron lo mismo), arreglar la causa raíz y evitar que se repita. Así se gestiona profesionalmente, con un responsable en cada paso:

1. **Atención al cliente** recibe la queja, abre la incidencia y devuelve los 10 € a Catalina.
2. **Equipo de IA** investiga la traza y encuentra la causa raíz.
3. **Equipo de IA** busca en todas las trazas el mismo patrón (precio dicho distinto del cobrado) para saber a cuántos clientes afectó.
4. **Equipo de backend**, responsable de la herramienta de búsqueda, cambia el campo para que devuelva siempre el precio final.
5. **Equipo de IA** ajusta las instrucciones del agente para que confirme el precio con lo que devuelve la reserva.
6. **Calidad (QA)** añade el caso a las pruebas automáticas y comprueba el arreglo antes de publicarlo.
7. **Todos** escriben un post-mortem (qué pasó y qué se aprende, sin buscar culpables), y atención al cliente avisa a los afectados.

Los dos arreglos, antes y después:

```
Herramienta, antes:   { "codigo": "AB0738", "aerolinea": "AeroBrisa", "precio": 85 }
Herramienta, después: { "codigo": "AB0738", "aerolinea": "AeroBrisa",
                        "precio_sin_tasas": 85, "tasas": 10, "precio_final": 95 }

Agente, antes:   "Cuando termines la reserva, informa al cliente
                  del vuelo y del precio."
Agente, después: "Cuando termines la reserva, informa al cliente
                  del vuelo y del precio_final que devuelve reservar_vuelo.
                  Nunca uses el precio de la búsqueda."
```

> **La traza es lo que permite a todos los equipos trabajar con datos en lugar de con opiniones.**

10. Evaluación

Observar no es evaluar. La traza nos dijo qué pasó; la evaluación comprueba, cada vez que cambiamos algo, que el agente lo hace bien. Para eso se usa un **golden set**: un conjunto de casos de prueba preparados por personas, con la respuesta correcta conocida. El caso de Catalina pasa a formar parte de él.

¿Quién comprueba cada cosa?

Qué se comprueba	Quién	Por qué
El precio que dice el agente coincide con el cobrado.	Código	Es una regla exacta: rápido, gratis y sin errores.
La respuesta es clara y amable.	LLM como juez	No hay regla exacta; otro modelo lo puntúa, pero hay que comprobar que acierta.
Cuál es la respuesta correcta de un caso nuevo.	Una persona	Lo más fiable y lo más caro; también sirve para comprobar al juez.

Al ejecutar un golden set de 20 casos antes del arreglo, 17 son correctos y uno de los fallos es el de Catalina: no se puede publicar. Después del arreglo, 20 de 20: ahora sí.

Esto es la evaluación **offline**, antes de publicar. Con clientes reales se hace también evaluación **online**: se revisa automáticamente una parte de las conversaciones y se recoge la opinión de los usuarios.

11. El ciclo completo

1. **Trabaja.** El agente atiende a clientes reales.
2. **Registra.** La observabilidad guarda cada tarea como una traza.
3. **Avisa.** Las métricas y las quejas indican que algo va mal.
4. **Investiga.** Con las trazas se encuentra la causa.
5. **Aprende.** El fallo se convierte en un caso del golden set.
6. **Corrige.** Se arregla la herramienta, el agente o los dos.
7. **Evalúa.** Se ejecuta el golden set antes de publicar.
8. **Publica.** Se publica el cambio, y vuelta a empezar.

> **No puedes evaluar lo que no observas, y observar sin evaluar solo te da datos sin criterio.**

12. Riesgos y ética

Observar un agente significa guardar lo que hacen y dicen los clientes, y eso exige responsabilidad.

- **Privacidad.** Los datos personales (nombre, correo, teléfono, tarjeta) deben ocultarse antes de guardarse en una traza. El RGPD lo exige.
- **Secretos.** Nunca deben quedar claves de API ni contraseñas en una traza.
- **Coste.** Guardar y evaluar todo cuesta dinero y espacio: hay que decidir qué se guarda y durante cuánto tiempo.
- **Falsa confianza.** Spans en verde y un juez sin comprobar pueden hacer creer que todo va bien.
- **Sesgos del juez.** Un LLM juez puede preferir respuestas largas o puntuar distinto según el idioma; por eso se comprueba con personas.

Las herramientas profesionales permiten ocultar o filtrar datos sensibles antes de enviarlos y definir cuánto tiempo se conservan las trazas. Conviene decidirlo desde el primer día.

13. Errores de concepto

- "Observabilidad es tener un panel con gráficas." No: un panel responde preguntas preparadas; observar permite responder preguntas nuevas.
- "Un log muy detallado ya es una traza." No: un log es una anotación suelta; una traza es una historia ordenada con causa y efecto.
- "Si todos los spans están en verde, el agente funcionó." No: verde significa que el código no se rompió.
- "Si falla, la culpa es del modelo." No siempre: puede ser una herramienta, un dato mal formado o las instrucciones.
- "Evaluar es poner un LLM como juez y ya." No: un juez sin comprobar puede equivocarse sin que nadie lo note.
- "La observabilidad solo sirve para arreglar fallos." No: también sirve para controlar costes, mejorar el agente, auditar decisiones y crear casos de prueba.

14. Conclusiones

Tu agente dice que todo ha ido bien. ¿Cómo lo sabes? **Con trazas, para saber qué pasó. Y con evaluación, para saber si estuvo bien.**



1. Las palabras del agente no son pruebas; la traza sí.
2. Verde no significa correcto.
3. Cada fallo encontrado se convierte en una prueba para que no vuelva a pasar.

15. Para seguir aprendiendo

Lecturas recomendadas (en inglés)

- Anthropic, *Building effective agents*: qué es un agente y sus patrones más comunes. anthropic.com/engineering/building-effective-agents
- OpenTelemetry, *Observability primer*: introducción oficial a logs, métricas, trazas y spans. opentelemetry.io/docs/concepts/observability-primer
- OpenTelemetry, *Semantic conventions for generative AI*: el vocabulario común gen_ai.*. opentelemetry.io/docs/specs/semconv/gen-ai
- Documentación de Langfuse: trazas y evaluaciones con ejemplos visuales. langfuse.com/docs
- Hamel Husain, *Your AI Product Needs Evals*: cómo evaluar productos de IA en la práctica. hamel.dev/blog/posts/evals

- Google (2010), *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*: el origen del modelo de trazas y spans.
- Yao et al. (2022), *ReAct: Synergizing Reasoning and Acting in Language Models*: el patrón de razonar y actuar en bucle. arxiv.org/abs/2210.03629

Glosario

Agente
Un modelo de lenguaje en un bucle, que decide pasos y usa herramientas para cumplir un encargo.

Observabilidad
La capacidad de reconstruir lo que hizo un sistema, incluso ante preguntas no previstas.

Instrumentar
Preparar un sistema para que registre lo que hace.

Log
Una anotación de algo que pasó en un momento concreto.

Métrica
Un número medido a lo largo del tiempo: duración, tokens, coste, errores.

Traza
El recorrido completo de una tarea, con su relación de causa y efecto.

Span
Un tramo de trabajo dentro de una traza, con nombre, duración, estado y datos.

Golden set
Un conjunto de casos de prueba con la respuesta correcta conocida.

LLM como juez
Usar un modelo para puntuar lo que no se puede comprobar con una regla exacta.

Causa raíz
El origen real de un fallo, no solo su síntoma.

Post-mortem
Un informe de qué pasó en una incidencia y qué se aprende, sin buscar culpables.

Guía de la masterclass «Evaluación y observabilidad de sistemas de agentes», impartida el 5 de noviembre de 2026 en el Bootcamp de Desarrollo de IA de Somos F5. © 2026 David Rojas Cruz · Licencia CC BY-NC-ND 4.0. Las trazas, clientas, aerolíneas, la agencia y los datos son ficticios, creados con fines didácticos.